

Flight Operations Dashboard — Design Explanation

1. Goal and users

The dashboard is for **aviation operations controllers** who need to answer three questions fast: *What is flying right now? What needs my attention? Where is this specific flight and when will it arrive?* Every layout and interaction choice below serves those questions, in that order of importance.

2. Layout and UX

The map is the hero. It takes the largest area of the screen; everything else supports it. The layout has three zones:

- **KPI strip (top):** Total, Active, Delayed and Arrived flights as large numbers with a colour-coded edge and an icon. A controller can read the state of operations in one glance. Delayed uses amber to signal “needs attention”.
- **Left panel – search, filters and flight list:** Callsign search, status, origin and destination filters sit above a scrollable list. Each row shows flight number, callsign, route, status, ETA and a thin progress bar, so the list works as a second, scannable view of the same data. A live “N of M flights” count and a one-click “Clear filters” reduce dead ends; an empty state on the map explains why nothing is shown.
- **Right panel – flight details:** Opens when a flight is selected (from the list, a marker, or a deep link such as `/flight/FL003`). It leads with the route (HYD → DEL) and progress, then the required fields in a two-column definition grid. `ESC` or the close button dismisses it.

Selecting a flight highlights its marker, draws the great-circle route (solid for the part already flown, dashed for what remains, with a soft halo), labels origin and destination, and flies the map to the aircraft. During playback the map follows the aircraft.

3. Visual design

A calm, low-chrome interface so that the colour in the data stands out. Neutral surfaces, a single sky-blue accent, and four status colours used consistently in markers, badges, KPIs and the legend: slate (scheduled), green (active), amber (delayed), blue (arrived). The type is Inter with tabular figures for times and counts. Spacing follows a 4/8 px rhythm, with 12 px radius cards and subtle elevation. **Dark mode** (persisted, follows the OS preference) swaps the whole token set and the basemap, which is useful in dim operations rooms.

4. Responsiveness

Width	Behaviour
Desktop (> 1100 px)	Three columns: list, map, details
Tablet (≤ 1100 px)	Map is full width; the flight list becomes a slide-in drawer (hamburger); details float over the map’s right edge so the aircraft stays visible; KPI hints are dropped to avoid wrapping
Phone (≤ 720 px)	KPIs in 2×2, details as a bottom sheet, speed selector hidden

The map uses a **ResizeObserver** so Leaflet re-measures whenever the layout changes.

5. Accessibility

Skip link and landmarks; every control has a visible label or **aria-label**; toggles use **aria-pressed**; the selected list item uses **aria-current**; counts are **aria-live**. Markers are keyboard-focusable and carry descriptive **alt/title** text. **Status is never conveyed by colour alone** — each badge pairs a glyph and the status word. Focus rings are always visible, text/background pairs are chosen for AA contrast in both themes, and **prefers-reduced-motion** disables animation.

6. Architecture

Angular 18 standalone components, strict TypeScript, **OnPush** everywhere.

- **Container + presentational components.** **DashboardComponent** is the only component that talks to services; it merges the service streams into one view-model rendered with the **async** pipe. Header, filters, list, details, map, KPI card and status badge are inputs/outputs only, so they are reusable and cheap to test.
- **FlightService** owns state with **BehaviorSubjects** and exposes derived, shared streams: `filteredFlights$, kpis$, selectedFlight$,` option lists for the filters, and playback state.

- **Time-driven model.** A flight is a *pure function of a simulation clock* (`resolveFlight(blueprint, now)`). Status (Scheduled / Active / Delayed / Arrived), position (great-circle interpolation), heading, altitude and speed all derive from it. Playback is therefore just “advance the clock”, and KPIs can never disagree with the map.
- **Reactive Forms** drive the filters (typed, debounced); **routing** provides deep links through a single `UrlMatcher` route so the map is never destroyed on selection.

7. Leaflet integration

Leaflet runs outside Angular’s zone so map events don’t cause app-wide change detection. Markers are `divIcon` aircraft glyphs rotated to the true heading and coloured by status, and are **updated in place** each tick rather than recreated.

`leaflet.markercluster` handles dense airports (toggleable), airports are an optional layer, a RainViewer radar overlay is optional, and the basemap switches with the theme.

8. Trade-offs and next steps

Data is mocked from JSON and positions are interpolated along great circles; a real feed would swap the `HttpClient` calls for a `WebSocket`/polling source with no UI change. Next steps I would take: virtual scrolling for thousands of flights, a timeline scrubber for playback, saved filter views, and Playwright end-to-end tests.